IMAGE PROCESSING

ANALYSIS OF JPEG COMPRESSION

HIRUSHI PREMARATHNA
200479D

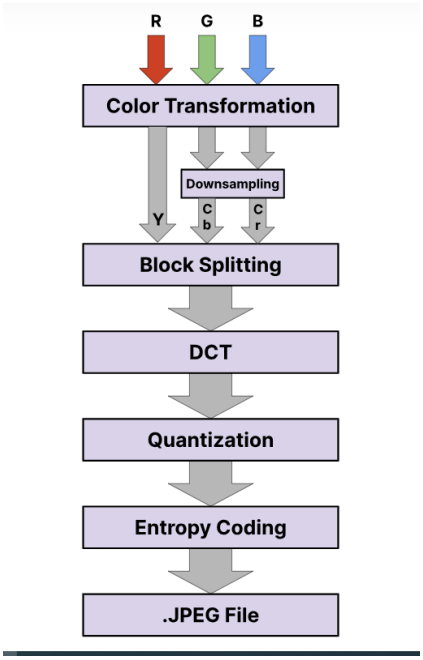
Introduction

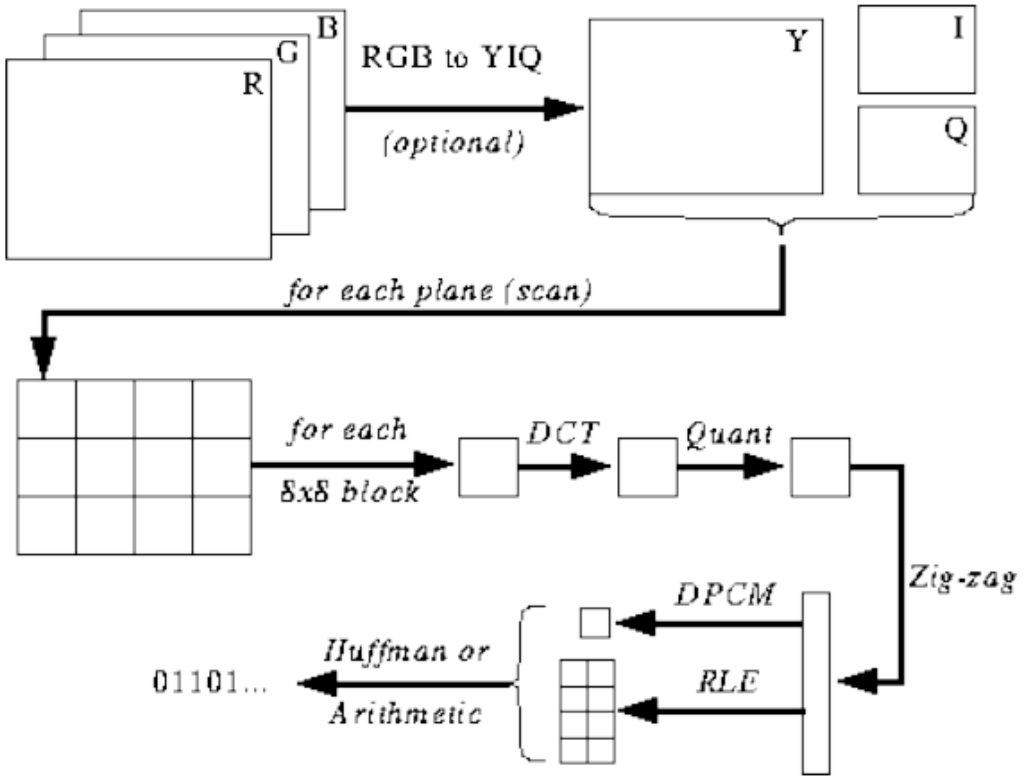
JPEG (Joint Photographic Experts Group) compression is a widely used method for reducing the size of digital images while maintaining an acceptable level of quality. This report will delve into the key processing steps involved in JPEG compression for a 24bpp (bits per pixel) image and provide examples to illustrate these steps.

JPEG is a lossy compression method, meaning it sacrifices some image quality to achieve smaller file sizes. This trade-off is essential for efficient storage and transmission of digital images.

The main steps of JPEG compression are as following,

1. Colour Space Conversion
2. Chroma Subsampling (Optional)
3. Discrete Cosine Transform (DCT)
4. Quantization
5. Zigzag Scanning
6. Run-Length Encoding (RLE)
7. Huffman Coding





The following pages will detail the stages of JPEG compression.

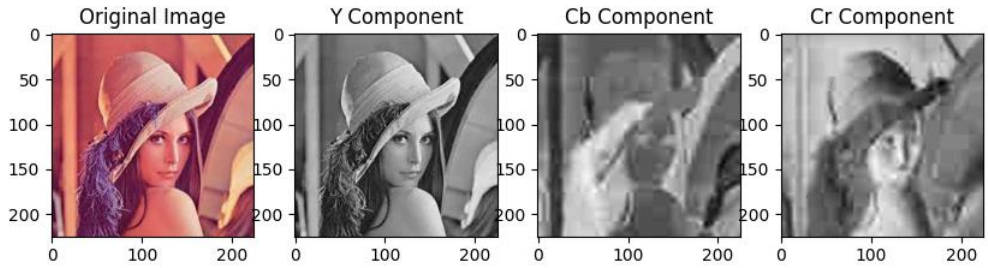
1. Colour Space Conversion

In a 24-bit per pixel (bpp) image, each pixel is typically represented in the RGB (Red, Green, Blue) colour space, which is highly intuitive for human perception. However, RGB is not the most efficient colour space for image compression. JPEG prefers to work in the YCbCr colour space, The Y component determines the brightness of the colour (also referred to as luminance or luma), while the U and V components determine the colour (also known as chroma). The U component refers to the amount of blue colour and the V component refers to the amount of red colour.

The conversion from RGB to YCbCr is achieved through the following linear equations:

Y= 16 + 65.738*R/256 + 129.057*G/256 + 25.064*B/256
Cb = 128-37.945*R/256 - 74.494*G/256 + 112.439*B/256
Cr = 128+112.439*R - 94.154*G/256 - 18.285*B/256

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
input_image = Image.open('images.jpg')
image_array = np.array(input_image)
def rgb_to_ycbcr(pixel):
    y= 16 + 65.738*pixel[0]/256 + 129.057*pixel[1]/256 + 25.064*pixel[2]/256
    cb = 128-37.945*pixel[0]/256 - 74.494*pixel[1]/256 + 112.439*pixel[2]/256
    cr = 128+112.439*pixel[0] - 94.154*pixel[1]/256 - 18.285*pixel[2]/256
    return y, cb, cr
ycbcr_image = np.apply_along_axis(rgb_to_ycbcr, 2, image_array)
y_channel = ycbcr_image[:, :, 0]
cb_channel = ycbcr_image[:, :, 1]
cr_channel = ycbcr_image[:, :, 2]
plt.figure(figsize=(10, 5))
plt.subplot(1, 4, 1)
plt.title("Original Image")
plt.imshow(input_image)
plt.subplot(1, 4, 2)
plt.title("Y Component")
plt.imshow(y_channel, cmap='gray')
plt.subplot(1, 4, 3)
plt.title("Cb Component")
plt.imshow(cb_channel, cmap='gray')
plt.subplot(1, 4, 4)
plt.title("Cr Component")
plt.imshow(cr_channel, cmap='gray')
print(y_channel,y_channel.shape)
print(cb_channel,cb_channel.shape)
print(cr_channel,cr_channel.shape)
plt.show()
```



The channel (Y) preserves most of the image's visual quality, while the colour channels (Cb and Cr) can be subsampled, further reducing the data size.

2. Chroma Subsampling (Optional)

Chroma subsampling is an optional step in the JPEG compression process, allowing for further reduction of data size. The rationale behind chroma subsampling is rooted in the fact that the human visual system is more sensitive to changes in brightness than colour. As a result, it's possible to reduce the resolution of the colour channels without significantly affecting perceived image quality.

Common subsampling ratios include 4:4:4, 4:2:2, and 4:2:0, where the first number represents the horizontal subsampling of the luma channel, the second number represents the horizontal subsampling of the Cb channel, and the third number represents the vertical subsampling of the Cr channel. For instance, in a 4:2:0 subsampling, the chroma channels are subsampled by a factor of 2 in both the horizontal and vertical directions.

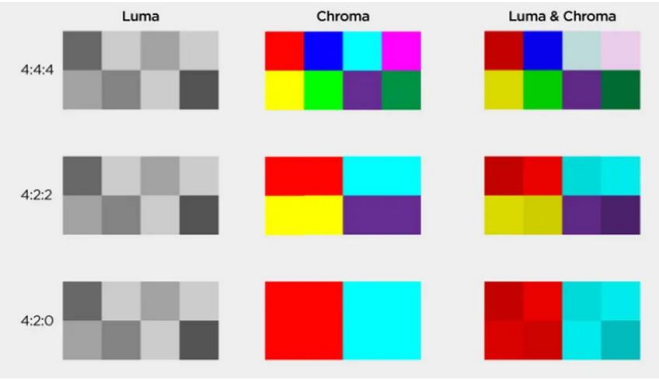
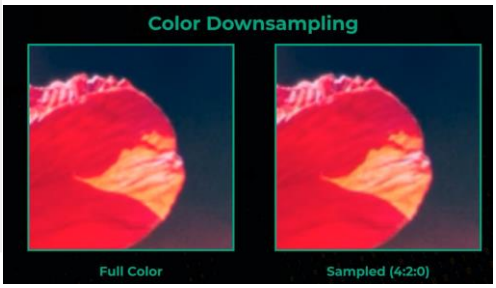


Fig. 1 — chroma subsampling formats examples [3]



The left side of image shows how down sampling works while other shows a magnified image how it affects.

This demonstrates how subsampling reduces the size of the chroma channels, making the image more compact for subsequent compression steps.

For the process of JPEG compression, I intended to use 4:4:4 method.

3. Discrete Cosine Transform (DCT)

After the colour space conversion and potential chroma subsampling, the next key step in the JPEG compression process is the Discrete Cosine Transform (DCT). The DCT is a mathematical technique used to convert spatial information into frequency information. By analysing the frequency

components of an image, JPEG can efficiently represent them, as many high-frequency components can be approximated to zero.

Block splitting should do prior to applying DCT. Each block in image is typically treated as an independent unit for processing. In JPEG compression, for instance, the 8x8 block size is a common choice. When the image dimensions are not exact multiples of the block size, partial blocks are created at the image's edges. Handling these partial blocks is crucial; methods like zero-padding or separate processing can be employed to ensure that every part of the image is effectively managed during compression.

After selecting a block, the values in the block should be shifted to centered around zero. This process involves subtracting 128 from each value, converting the range from [0, 255] to [-128, 127]. This step reduces dynamic range requirements in the subsequent DCT processing.

```
import numpy as np

def recenter_matrix(component):
    component = np.array(component)
    centered_matrix = component - 128
    return centered_matrix
#samply 8*8 block from the y component
component = np.array([
    [154.12, 154.12, 153.26, 153.26, 152.71, 152.71, 152.71, 151.85],
    [153.26, 153.26, 153.26, 152.40, 152.71, 151.85, 151.85, 151.75],
    [153.26, 152.40, 152.40, 152.40, 151.85, 151.85, 150.89, 150.89],
    [152.40, 151.54, 151.54, 151.85, 150.99, 150.99, 150.03, 150.03],
    [150.99, 150.99, 150.99, 150.13, 150.13, 149.17, 149.17, 149.17],
    [150.13, 150.13, 150.13, 149.27, 149.17, 148.31, 148.31, 148.12],
    [150.03, 149.17, 149.17, 149.17, 148.31, 148.31, 147.26, 147.26],
    [149.17, 149.17, 149.17, 148.31, 148.31, 147.26, 147.26, 147.57]
])
print(recenter_matrix(component))
```

Resulted output:

```
[[26.12 26.12 25.26 25.26 24.71 24.71 24.71 23.85]
 [25.26 25.26 25.26 24.4  24.71 23.85 23.85 23.75]
 [25.26 24.4  24.4  24.4  23.85 23.85 22.89 22.89]
 [24.4  23.54 23.54 23.85 22.99 22.99 22.03 22.03]
 [22.99 22.99 22.99 22.13 22.13 21.17 21.17 21.17]
 [22.13 22.13 22.13 21.27 21.17 20.31 20.31 20.12]
 [22.03 21.17 21.17 21.17 20.31 20.31 19.26 19.26]
 [21.17 21.17 21.17 20.31 20.31 19.26 19.26 19.57]]
```

Like that need to perform for all 3 components of YCbCr.

Next for each component, DCT coefficient should be found. The following figure shows the how to find each element DCT coefficient.

$$G_{u,v} = \frac{1}{4}\alpha(u)\alpha(v)\sum_{x=0}^{7}\sum_{y=0}^{7} g_{x,y}\cos\left[\frac{(2x+1)u\pi}{16}\right]\cos\left[\frac{(2y+1)v\pi}{16}\right]$$

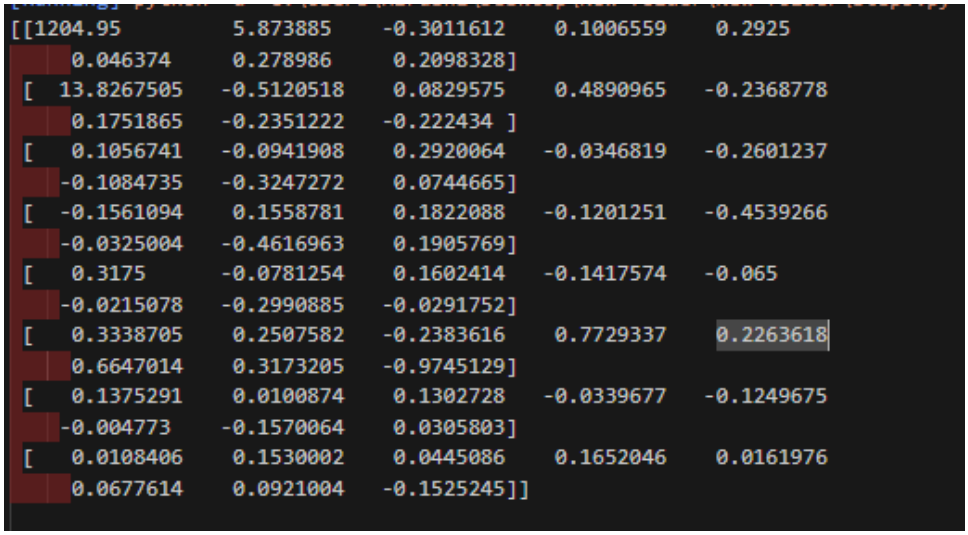
where

- u is the horizontal spatial frequency, for the integers $0 \leq u < 8$.
- v is the vertical spatial frequency, for the integers $0 \leq v < 8$.
- $\alpha(u) = \begin{cases} \frac{1}{\sqrt{2}}, & \text{if } u = 0 \\ 1, & \text{otherwise} \end{cases}$ is a normalizing scale factor to make the transformation orthonormal
- $g_{x,y}$ is the pixel value at coordinates (x, y)
- $G_{u,v}$ is the DCT coefficient at coordinates (u, v).

To perform DCT, I selected a 8*8 block of the Y component in the image for the example.

```
import numpy as np
import math
def dct(component):
    n, m = 8, 8
    dct_result = np.zeros((n, m))
    for i in range(m):
        for j in range(n):
            ci = 1 / math.sqrt(m) if i == 0 else math.sqrt(2 / m)
            cj = 1 / math.sqrt(n) if j == 0 else math.sqrt(2 / n)
            sum_dct = 0
            for x in range(m):
                for y in range(n):
                    dct1 = component[x][y] * math.cos((2 * x + 1) * i * np.pi
/ (2 * m)) * math.cos((2 * y + 1) * j * np.pi / (2 * n))
                    sum_dct += dct1
            dct_result[i][j] = round(ci * cj * sum_dct, 7)
    return dct_result
component = np.array([
    [26.12, 26.12, 25.26, 25.26, 24.71, 24.71, 24.71, 23.85],
    [25.26, 25.26, 25.26, 24.4, 24.71, 23.85, 23.85, 23.75],
    [25.26, 24.4, 24.4, 24.4, 23.85, 23.85, 22.89, 22.89],
    [24.4, 23.54, 23.54, 23.85, 22.99, 22.99, 22.03, 22.03],
    [22.99, 22.99, 22.99, 22.13, 22.13, 21.17, 21.17, 21.17],
    [22.13, 22.13, 22.13, 21.27, 21.17, 20.31, 20.31, 20.12],
    [22.03, 21.17, 21.17, 21.17, 20.31, 20.31, 19.26, 19.26],
    [21.17, 21.17, 21.17, 20.31, 20.31, 19.26, 19.26, 19.57]
])
dct_matrix = dct(component)
np.set_printoptions(precision=7, suppress=True)
print(dct_matrix)
```

Output



```
[[1204.95        5.873885   -0.3011612   0.1006559   0.2925
    0.046374    0.278986    0.2098328]
 [  13.8267505  -0.5120518   0.0829575   0.4890965  -0.2368778
    0.1751865  -0.2351222  -0.222434 ]
 [   0.1056741  -0.0941908   0.2920064  -0.0346819  -0.2601237
   -0.1084735  -0.3247272   0.0744665]
 [  -0.1561094   0.1558781   0.1822088  -0.1201251  -0.4539266
   -0.0325004  -0.4616963   0.1905769]
 [   0.3175     -0.0781254   0.1602414  -0.1417574  -0.065
   -0.0215078  -0.2990885  -0.0291752]
 [   0.3338705   0.2507582  -0.2383616   0.7729337   0.2263618
    0.6647014   0.3173205  -0.9745129]
 [   0.1375291   0.0100874   0.1302728  -0.0339677  -0.1249675
   -0.004773   -0.1570064   0.0305803]
 [   0.0108406   0.1530002   0.0445086   0.1652046   0.0161976
    0.0677614   0.0921004  -0.1525245]]
```

The DCT coefficients provide a representation of the spatial frequency components of the block. Note that many coefficients are close to zero, indicating that they contain less visual information and can be quantized to further reduce the data size.

4. Quantization

Quantization is the process of reducing the precision of the DCT coefficients by dividing them by a set of quantization values, also known as quantization tables. These tables are predefined and have different values for the luminance (Y) and chrominance (Cb and Cr) components. The quantization process introduces loss of data precision and contributes significantly to the lossy nature of JPEG compression. Quantization matrix controls the compression ratio.

Quantization is done by dividing each DCT coefficient by the corresponding value in the quantization table and rounding to the nearest integer. This results in most coefficients being reduced to zero or very small values.

$$B_{j,k} = \text{round}\left(\frac{G_{j,k}}{Q_{j,k}}\right) \text{ for } j = 0, 1, 2, \ldots, 7; k = 0, 1, 2, \ldots, 7$$

Where:

- **B** = The quantized DCT coefficients
- **G** = The DCT coefficients
- **B** = The quantization matrix

Each of the Y (luminance), Cb (chroma blue), and Cr (chroma red) channels are calculated separately. There are two quantization tables in a JPEG image. One for the luminance channel and one for the chrominance channels.

For example, applying quantization to the DCT coefficients shown above using a quantization table for luminance (Y) might yield:

```
import numpy as np
def quantize_dct_coefficients(dct_coefficients, quantization_matrix):
    quantized_coefficients = np.round(dct_coefficients / quantization_matrix)
    return quantized_coefficients
luminance_qm = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
])
chrominance_qm = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
])
dct_coefficients = np.array([
    [1204.95, 5.873885, -0.3011612, 0.1006559, 0.2925, 0.046374, 0.278986,
0.2098328],
    [13.8267505, -0.5120518, 0.0829575, 0.4890965, -0.2368778, 0.1751865, -
0.2351222, -0.222434],
    [0.1056741, -0.0941908, 0.2920064, -0.0346819, -0.2601237, -0.1084735, -
0.3247272, 0.0744665],
    [-0.1561094, 0.1558781, 0.1822088, -0.1201251, -0.4539266, -0.0325004, -
0.4616963, 0.1905769],
    [0.3175, -0.0781254, 0.1602414, -0.1417574, -0.065, -0.0215078, -
0.2990885, -0.0291752],
    [0.3338705, 0.2507582, -0.2383616, 0.7729337, 0.2263618, 0.6647014,
0.3173205, -0.9745129],
    [0.1375291, 0.0100874, 0.1302728, -0.0339677, -0.1249675, -0.004773, -
0.1570064, 0.0305803],
    [0.0108406, 0.1530002, 0.0445086, 0.1652046, 0.0161976, 0.0677614,
0.0921004, -0.1525245]
])
quantized_luminance = quantize_dct_coefficients(dct_coefficients,
luminance_qm)
print("Quantized Luminance:")
print(quantized_luminance)
```

```
Quantized Luminance:
[[75.  1. -0.  0.  0.  0.  0.  0.]
 [ 1. -0.  0.  0. -0.  0. -0. -0.]
 [ 0. -0.  0. -0. -0. -0. -0.  0.]
 [-0.  0.  0. -0. -0. -0. -0.  0.]
 [ 0. -0.  0. -0. -0. -0. -0. -0.]
 [ 0.  0. -0.  0.  0.  0.  0. -0.]
 [ 0.  0.  0. -0. -0. -0. -0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0. -0.]]
```

output

The quantized coefficients reveal the impact of quantization, as most values are now zero. This quantization step plays a crucial role in reducing data size while allowing for potential image quality adjustments.

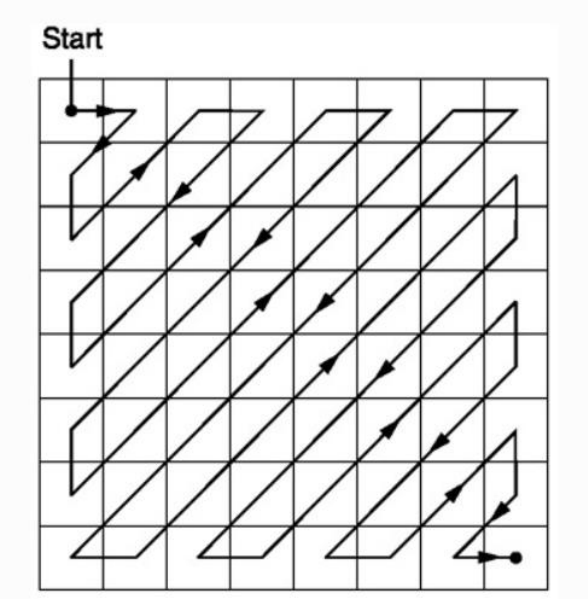
5. Zigzag Scanning

After the DCT coefficients are quantized, the data is rearranged using zigzag scanning. This step reorders the coefficients in a manner that optimizes their representation for entropy coding, making it easier to identify runs of zeros and compress the data more effectively.

Zigzag scanning converts the 2D array of quantized coefficients into a 1D array by following a zigzag pattern. This ensures that non-zero coefficients are grouped together and makes run-length encoding (RLE) more efficient.

Let's consider an example. Starting from the top left of an 8x8 block, the coefficients are zigzag-scanned as follows:

Zigzag graph




```
import numpy as np
def zigzag_scan(array):
    rows, cols = array.shape
    result = np.zeros(rows * cols, dtype=int)
```

```
    i, j = 0, 0
    for k in range(rows * cols):
        result[k] = array[i, j]
        if (i + j) % 2 == 0:  # Moving up
            if i > 0 and j < cols - 1:
                i -= 1
                j += 1
            elif j < cols - 1:
                j += 1
            else:
                i += 1
        else:  # Moving down
            if i < rows - 1 and j > 0:
                i += 1
                j -= 1
            elif i < rows - 1:
                i += 1
            else:
                j += 1
    return result
sample_array = np.array([
    [75, 1, 0, 0, 0, 0, 0, 0],
    [1, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0]
])
zigzag_result = zigzag_scan(sample_array)
print(", ".join(map(str, zigzag_result)))

Output:
```

75, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

6. Run-Length Encoding (RLE)

RLE is a simple yet powerful technique for compressing sequences of data that contain long runs of the same value. In JPEG compression, RLE is used to encode the zigzag-scanned DCT coefficients efficiently, taking advantage of the numerous zeros introduced by quantization.

RLE works by identifying runs of zeros and encoding them as pairs of (run-length, value). The run-length works by removing zeros from the data sequence and recording how many zeros came before the next item in the series.

```
import numpy as np
def rle_encode(data):
```

```
    encoded_data = []
    count = 1
    for i in range(1, len(data)):
        if data[i] == data[i - 1]:
            count += 1
        else:
            encoded_data.append((data[i - 1], count))
            count = 1
    encoded_data.append((data[-1], count))
    return encoded_data
original_data = [75, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
encoded_data = rle_encode(original_data)
print(encoded_data)
```

Output:

[(75, 1), (1, 2), (0, 61)]

The RLE-encoded data is significantly more compact and can be easily decoded to reconstruct the original sequence.

7. Huffman Coding

The final step in JPEG compression is Huffman coding, which is applied to the RLE-encoded data. Huffman coding is a variable-length prefix coding technique that assigns shorter codes to more frequently occurring values. In the context of JPEG, Huffman coding is used to further reduce the size of the data by encoding the (run-length, value) pairs efficiently.

Huffman tables are used to map the (run-length, value) pairs to binary codes. Shorter codes are assigned to frequently occurring values, and longer codes to less frequent values.

For example, the (run-length, value) pairs from the RLE-encoded data:

[(1, 100), (7, 0), (24, 0)]

```
class Node:
    def __init__(self, symbol, frequency):
        self.symbol = symbol
        self.frequency = frequency
        self.left = None
        self.right = None
def build_huffman_tree(freq_table):
    nodes = [Node(symbol, freq) for symbol, freq in freq_table]
    while len(nodes) > 1:
        nodes.sort(key=lambda node: node.frequency)
        left = nodes.pop(0)
        right = nodes.pop(0)
```

```
        parent = Node(None, left.frequency + right.frequency)
        parent.left = left
        parent.right = right
        nodes.append(parent)
    return nodes[0]
def build_huffman_codes(root, prefix="", code={}):
    if root:
        if root.symbol is not None:
            code[root.symbol] = prefix
        build_huffman_codes(root.left, prefix + "0", code)
        build_huffman_codes(root.right, prefix + "1", code)
    return code
def encode_with_huffman(data, encoding_table):
    encoded_data = ""
    for symbol, freq in data:
        encoded_data += encoding_table[symbol] * freq
    return encoded_data
rle_data = [(75, 1), (1, 2), (0, 61)]
freq_table = rle_data
huffman_root = build_huffman_tree(freq_table)
huffman_encoding = build_huffman_codes(huffman_root)
encoded_data = encode_with_huffman(rle_data, huffman_encoding)
print(encoded_data)

Output: 000101111111111111111111111111111111111111111111111111111111111111111
```

The Huffman-coded data is the final representation of the compressed image, and it can be stored or transmitted more efficiently than the original image data.

In conclusion, JPEG compression transforms a 24bpp image through a series of processing steps, from colour space conversion to Huffman coding. The result is a compact representation of the image that balances data size with acceptable image quality, making JPEG a widely used format for various applications, from digital photography to web graphics.

After compression ended JPEG decompression procedure is done to display the image to the user.